

AI Lab Practical Exam

Complete Code Explanations — All 6 Programs

Code	Topic	Algorithm/Concept
Code 1	Graph Traversal	DFS & BFS
Code 2	8-Puzzle Solver	A* Search Algorithm
Code 3	Minimum Spanning Tree	Prim's Algorithm
Code 4	N-Queens Problem	Backtracking & Branch and Bound
Code 5	Customer Chatbot	Rule-Based / Keyword Matching
Code 6	Expert System	Knowledge Base + Inference Engine

Code 1 — Graph Traversal: DFS & BFS

The Graph Class & `__init__`

Sets up the graph structure with vertices and an adjacency list.

```
self.vertices = vertices
self.adj_list = {i: [] for i in range(vertices)}
# After init with 4 nodes: {0: [], 1: [], 2: [], 3: []}
```

- **vertices** = total number of nodes (e.g. 4 means nodes 0,1,2,3)
- **adj_list** = dictionary where each node has a list of its neighbors

`add_edge(source, destination)`

Draws a one-way arrow from source to destination. After all edges added:

```
adj_list = {0: [1,2], 1: [2], 2: [0,3], 3: [3]}
```

■ **Say to examiner:** *This is a directed graph. `add_edge` adds a connection from source to destination in the adjacency list.*

`dfs(start)` — Depth First Search

Analogy: Maze solving — go as deep as possible down one path before backtracking.

```
visited[vertex] = True      # mark current node
traversal.append(vertex)    # record it
for neighbour in adj_list[vertex]:
    if not visited[neighbour]:
        dfs_recursive(neighbour) # go DEEP first
```

Step-by-step from vertex 2:

```
Start at 2 → visit 2
Go to 0 → visit 0
Go to 1 → visit 1
Go to 2 → already visited, skip
Back to 2 → go to 3 → visit 3
Result: [2, 0, 1, 3]
```

■ **Say to examiner:** *DFS uses recursion and a visited array. It goes deep into one branch completely before moving to the next.*

`bfs(start)` — Breadth First Search

Analogy: Ripples in water — visit all neighbors first, then their neighbors.

```
queue = deque([start])      # start with first node
vertex = queue.popleft()     # take from front
for neighbour in adj_list[vertex]:
    if not visited[neighbour]:
        queue.append(neighbour) # add to back
```

Step-by-step from vertex 2:

```
Queue: [2] → visit 2, add 0,3
Queue: [0,3] → visit 0, add 1
Queue: [3,1] → visit 3
Queue: [1] → visit 1
```

Result: [2, 0, 3, 1]

■ **Say to examiner:** *BFS uses a queue (FIFO). It visits all nodes level by level — all neighbors first, then their neighbors.*

Key Difference: DFS vs BFS

	DFS	BFS
Strategy	Go deep first	Go wide first
Data Structure	Stack (via recursion)	Queue
Analogy	Maze solving	Ripples in water

Code 2 — 8-Puzzle Solver: A* Algorithm

A 3x3 grid with numbers 1-8 and one blank (`_`). You slide tiles to reach the goal arrangement.

PuzzleNode Class

```
self.g = g      # steps taken so far
self.h = h      # estimated steps remaining (heuristic)
self.f = g + h  # total cost (lower = better)
```

- **g** = how many moves made so far (actual cost)
- **h** = Manhattan distance — estimate of remaining moves
- **f = g + h** → A* always picks the node with lowest f

manhattan_distance(state, goal_positions)

For each tile: $|current_row - goal_row| + |current_col - goal_col|$. Add all values together = total heuristic distance.

■ **Say to examiner:** *Manhattan distance is our heuristic. It estimates how far each tile is from its goal position by counting horizontal + vertical moves needed.*

find_blank(state)

Scans the 3x3 grid to find where 0 (the empty space) is located. Returns its (row, col) position.

generate_neighbors(state)

Tries to move the blank Up, Down, Left, Right. Returns all valid new board states after each possible move.

reconstruct_path(node)

Traces back from goal to start using parent pointers, then reverses to give step-by-step solution.

a_star() — Main Algorithm

1. Start with initial state in open_list (min-heap)
2. Pick node with lowest f value
3. If it's the goal → return the path
4. Generate neighbors, calculate their g and h
5. Push valid neighbors into heap
6. Repeat until goal found

■ **Say to examiner:** *A* is an informed search. It uses $f = g + h$ to always explore the most promising path first, making it faster than BFS or DFS for puzzles.*

Code 3 — Minimum Spanning Tree: Prim's Algorithm

What is MST? Connect all nodes with minimum total edge weight, using exactly (vertices - 1) edges, with no cycles.

add_edge(u, v, weight)

```
self.adj_list[u].append((weight, v))
self.adj_list[v].append((weight, u)) # undirected = both ways
```

Since this is an undirected graph, we add the edge in both directions.

prim_mst(start=0) — Step by Step

1. Start from node 0
2. Use a min-heap (always pick cheapest edge next)
3. Mark node as visited
4. Add all unvisited neighbors to heap
5. Pick minimum weight edge from heap
6. Repeat until (vertices-1) edges selected

Example trace with given graph:

```
Start: Node 0
Pick edge 0→1 (cost 2) ✓
Pick edge 1→2 (cost 3) ✓
Pick edge 1→4 (cost 5) ✓
Pick edge 0→3 (cost 6) ✓
MST complete! Total cost = 16
```

■ **Say to examiner:** *Prim's algorithm builds the MST greedily. It always picks the minimum weight edge connecting a visited node to an unvisited node, using a min-heap for efficiency.*

Key Terms

- **Min-Heap:** A data structure that always gives you the smallest element first
- **Visited array:** Prevents revisiting nodes and forming cycles
- **MST:** Minimum Spanning Tree — connects all nodes with least total cost

Code 4 — N-Queens Problem: Backtracking & Branch and Bound

Place N queens on an NxN board so no two queens attack each other (no same row, column, or diagonal).

is_safe(row, col) — Backtracking version

Checks 3 things before placing a queen:

- 1. Same row? → scan left (no queen in that row)
- 2. Upper diagonal? → scan up-left
- 3. Lower diagonal? → scan down-left

backtrack(col) — Places queens column by column

```
Try row 0 in col 0 → safe? place queen → go to col 1
Try row 0 in col 1 → safe? no → try row 1...
...
If col == n → solution found! save it
If stuck → remove queen (board[row][col] = 0) → try next row
```

■ Say to examiner: Backtracking tries all possibilities and undoes wrong choices. When we hit a dead end, we backtrack and try a different path.

Branch and Bound — Faster version

Instead of scanning the whole board, uses 3 boolean arrays for O(1) conflict checking:

```
rows[row] # is this row occupied?
upper_diagonal[row + col] # is this upper diagonal occupied?
lower_diagonal[n-1+col-row] # is this lower diagonal occupied?
```

■ Say to examiner: Branch and Bound is smarter — it uses boolean arrays to check conflicts in O(1) time, pruning invalid paths early. Much faster than backtracking for large N.

Backtracking vs Branch and Bound

	Backtracking	Branch and Bound
Conflict Check	Scan whole board	O(1) with arrays
Speed	Slower	Faster
Memory	Less	Slightly more
Same results?	Yes	Yes

Code 5 — Customer Support Chatbot

A **rule-based chatbot** — no AI or ML, just if-else keyword matching on user input.

get_response(user_input)

```
message = user_input.lower().strip()    # normalize input

if "price" in message:      → reply about pricing
if "order" in message:     → reply about orders
if "return" in message:    → reply about returns
if "delivery" in message:  → reply about shipping
```

First it normalizes input (lowercase, strip spaces), then checks for keywords to pick the right response.

■ **Say to examiner:** *This is a rule-based chatbot. It uses keyword matching with if-else conditions. It's simple but effective for fixed-domain queries like customer support.*

main() — Chat Loop

```
Print welcome message
Loop forever:
    Take user input
    Get response from get_response()
    Print response
    If user says bye/exit → break loop
```

Keywords Handled

Keyword	Response Topic
hello / hi / hey	Welcome message
product / item	Product catalog
price / cost	Pricing info
order / track	Order tracking
delivery / shipping	Delivery time
return / refund	Return policy
payment	Payment methods
contact / support	Contact details
bye / exit / quit	Goodbye message

Code 6 — Expert System: Information Management

A program that mimics a human expert's decision making using a **Knowledge Base + Inference Engine + Explanation Facility**.

KNOWLEDGE_BASE

A list of 4 systems, each with conditions (rules) and an explanation:

```
{ "name": "Library Management System",  
  "conditions": { "data_type": "books", "users": "many",  
                  "security": "medium", "access": "search"},  
  "explanation": "Chosen for book records, multiple users..." }
```

ask_user()

Asks the user 4 questions and collects answers into a facts dictionary:

- What type of data? (books / medical / student / documents)
- How many users? (few / many)
- Security level? (low / medium / high)
- Main operation? (search / update / store)

infer_system(facts) — The Inference Engine

```
For each rule in knowledge base:  
    Count how many conditions match user's answers (score)  
    Track best matching rule  
Return the rule with highest score
```

■ **Say to examiner:** *The Inference Engine compares user input against all rules in the Knowledge Base and returns the best matching system using a scoring mechanism.*

explain_decision() — Explanation Facility

Prints why that system was recommended — which conditions matched and the justification.

Example Run

```
User answers: data_type=books, users=many,  
              security=medium, access=search  
→ Matches Library Management System (score: 4/4)  
→ Recommended! ✓
```

■ **Say to examiner:** *The expert system has 3 parts: Knowledge Base (stores rules), Inference Engine (matches user input using scoring), and Explanation Facility (explains why a system was recommended).*

Quick Reference — All 6 Codes

Code	Algorithm	Key Concept	One-liner for Examiner
1	DFS & BFS	Graph Traversal	DFS goes deep first (recursion/stack); BFS goes wide first (queue)
2	A* Search	8-Puzzle Solver	$f = g + h$; always picks most promising node using heuristic
3	Prim's	Min Spanning Tree	Greedily picks min-weight edge; uses min-heap for efficiency
4	Backtracking & B&B	N-Queens	Backtrack undoes wrong choices; B&B uses arrays for $O(1)$ checks
5	Rule-Based	Chatbot	Keyword matching with if-else; no ML, simple and domain-specific
6	Inference Engine	Expert System	Scores rules against user facts; recommends best matching system

Common One-Line Answers

- **Why adjacency list?** Memory efficient for sparse graphs compared to a matrix.
- **Why visited array?** To avoid visiting the same node twice and prevent infinite loops.
- **What is deque?** Double-ended queue from Python's collections module, efficient `popleft()` for BFS.
- **What is heuristic?** An estimate used to guide search — Manhattan distance in A*.
- **What is greedy?** Always picking the locally best option (Prim's picks minimum edge each step).
- **What is backtracking?** Try all options; undo (backtrack) when a path fails.
- **What is inference engine?** The reasoning part of an expert system that matches facts to rules.

Good luck on your Practical Exam! ■